

Mergesort

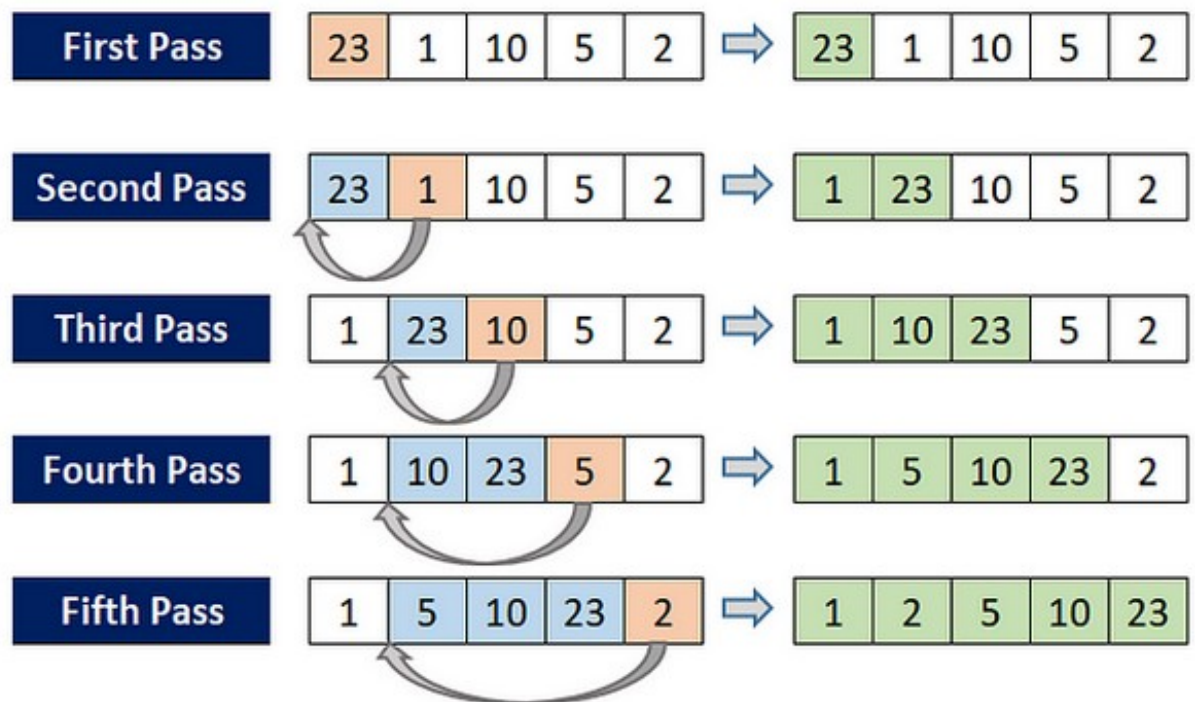
For additional reading and resources, you can refer to

<https://jeffe.cs.illinois.edu/teaching/algorithms/book/00-intro.pdf> and

<https://jeffe.cs.illinois.edu/teaching/algorithms/book/01-recursion.pdf>

Review from 12A Lab:

In our lecture on Tuesday morning, we covered selection sort. In lab, you were given insertion sort to implement. Insertion sort is different from selection sort! In selection sort, we always select the smallest value and move it to the beginning of our list. In insertion sort, we select values in the order they appear in the input array and when we move the value, we need to place it into the correct position on the "sorted" side of the array. Here is a visual representation of the algorithm:



There are some key steps in insertion sort:

1. We need to iterate through each of the elements in the array
2. For each element, we need to move it to the left side of our array (the "sorted" portion) and insert it into the correct position on the sorted side

For the implementation, we will follow the steps from the lab.

Insertion Sort Implementation

From the lab, we got the following instructions:

In Insertion sort, given an unsorted list `lst`, we first create a new empty list `res`, and everytime you remove one element from `lst` and put it in the right index of `res` so that `res` is always sorted. Then return `res`.

```
# Insertion Sort Implementation
def insertion_sort(lst):
    res = []

    for i in range(len(lst)):
        # find where we need to insert
        index = 0
        for j in range(len(res)):
            # check whether the number in res is < lst[i]
            if res[j] < lst[i]:
                index = index + 1

        # insert the element
        if index == 0: # beginning
            res = [lst[i]] + res
        elif index == len(res): # end
            res = res + [lst[i]]
        else:
            res = res[0:index] + [lst[i]] + res[index:]

    return res

l_unsorted = [1, 6, 2, 7, 8, 3]
print(l_unsorted)

l_sorted = insertion_sort(l_unsorted)
print(l_sorted)

[1, 6, 2, 7, 8, 3]
[1, 2, 3, 6, 7, 8]
```

Yesterday, we talked about quicksort and how to write it.

As a reminder, the high level idea for quick sort is:

1. Choose a pivot element from the array
2. Partition the array into three subarrays, containing elements smaller than pivot, pivot itself, and elements larger than pivot
3. recursively quick sort last and first subarrays

Today, we will cover a different sorting algorithm which is also a divide and conquer algorithm like quicksort: mergesort.

Divide and Conquer

Divide and conquer is a general, three-step pattern for algorithms:

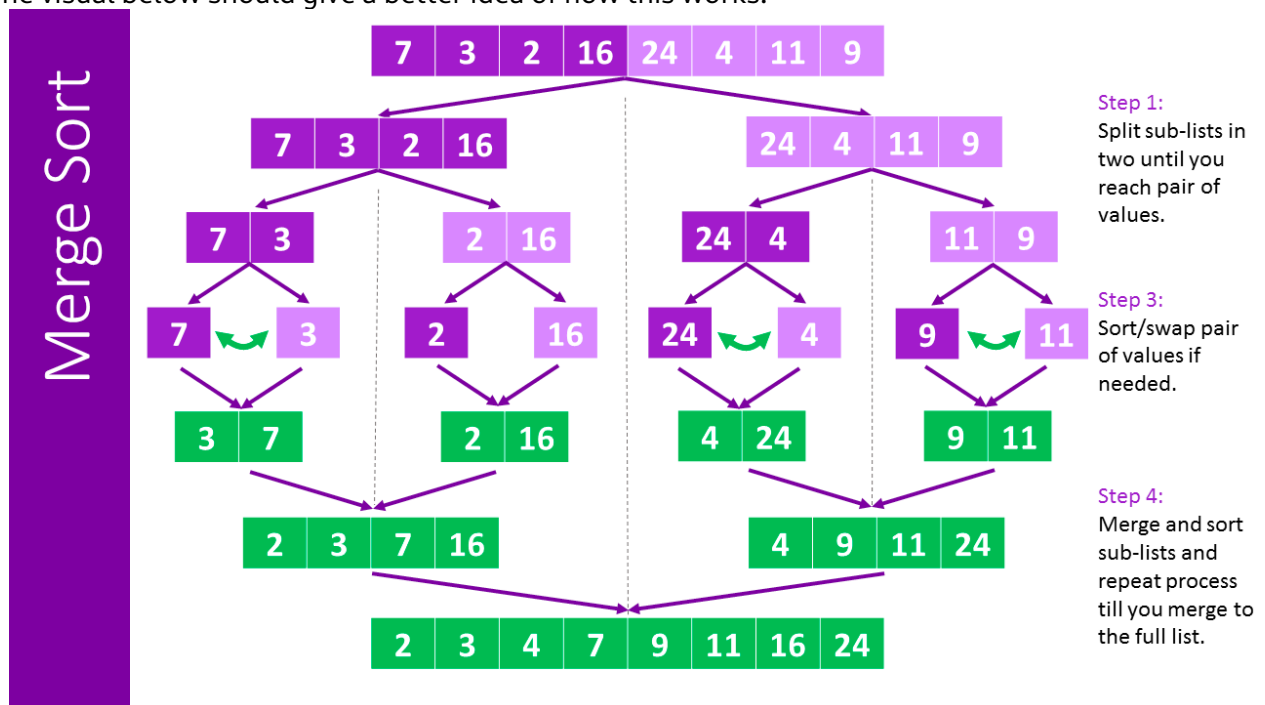
1. Divide the given instance of the problem into several independent smaller instances of exactly the same problem
2. Delegate each smaller instance to recursion
3. Combine the solutions for the smaller instances into the final solution for the given instance

This should be similar to a lot of recursive functions we have seen already.

How does Mergesort work?

Mergesort is one of the most efficient sort algorithms. It works by splitting the input array until we reach arrays of individual elements, and then merge all these individual arrays back together in sorted order.

The visual below should give a better idea of how this works:



In merge sort, we need to find the midpoint of the array and break the array in half using that index. What was another algorithm where we had to find the midpoint?

#

How did we find the midpoint in that algorithm?

#

Next, we will need to create a left array that will contain everything to the left of the midpoint and a right array that will contain everything to the right of the midpoint. What could we use to do this?

```
#
```

Finally, if we have two arrays, we need to merge these two arrays together. Let's think about how we can merge two separate arrays. Discuss with your neighbor and see if you can think through what sort of code you would write for this.

```
#
```

Mergesort Implementation

```
def merge(left, right, res): # merge left and right and store the  
result in res  
    pass  
  
def merge_sort(arr):  
    pass  
  
l_unsorted = [5, 7, 1, 8, 12, 4]  
  
merge_sort(l_unsorted)  
print(l_unsorted)  
[1, 4, 5, 7, 8, 12]
```

Mergesort Time Complexity

What is the worst case time complexity for merging two arrays?

```
# time complexity on board
```

Built-in Python Sort

The past few days, we have been learning about different sorting algorithms. However, there is a more simple way to sort in Python:

```
l_unsorted = [5, 1, 8, 3, 9, 2, 4, 1]  
  
l_sorted = sorted(l_unsorted)  
print(l_sorted)
```

If this function exists, why did we learn all those sorting algorithms?

The goal of learning sorting algorithms is to teach you to THINK, not just copy code that you're given. When you code, you will always get problems where you will have inputs that differ slightly from what you're used to. As a programmer, the goal is to think through how to solve those slightly different problems.

One example is that in lab, you've already experienced other implementations of some of the algorithms we have gone through.

